

第一章、 概论

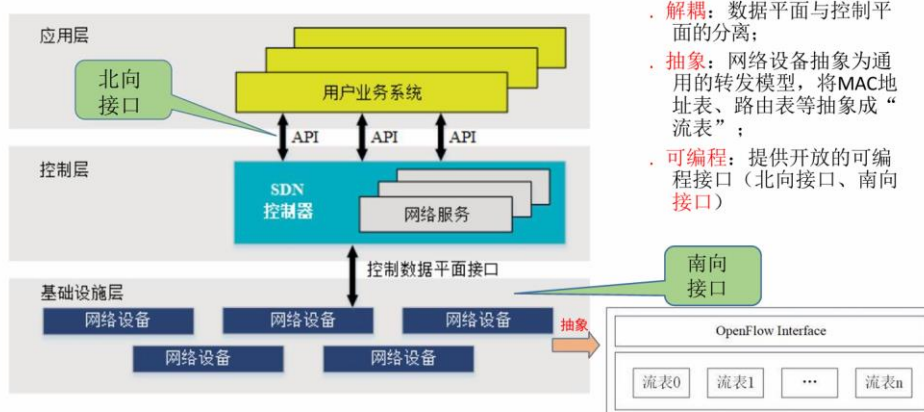
1.软件产品生命周期?

计划分析\设计\实现\测试\运维

网络系统的生命周期

规划设计|部署实施|运行维护|需求调研

SDN的三层网络架构



SDN的发展

- 第一阶段-SDN的发展初期 (2006-2011) : 提出了SDN的主要架构、设计思想和实现技术, 属于学术界的研究与试验初创阶段, 引起了产业界的关注;
- 第二阶段-SDN的发展中期 (2012至今) : 产业界开始普遍接受SDN的思想。SDN演变成以学院派、网络创新公司、传统网络巨头和运营商为代表的不同技术流派, 同时, 出现了更多的SDN控制器、网络操作系统、商用产品和应用案例

2.

SDN的标准化组织

开放网络基金会 (ONF) (2011)

因特网工程任务组 (IETF) (1985)

国际电信联盟电信标准化部门 (ITU-T) (1993)

欧洲电信标准化协会 (ETSI) (1988)

ONF 主要工作:

OpenFlow、SDN 白皮书、OF-Config 协议

OF-Config 是一个 OpenFlow 交换机配置协议，也是 ONF 主推的 OpenFlow 补充协议，很好的填补了 OpenFlow 协议之外的交换机运维配置等内容。它提供了开放接口用于控制和配置 OpenFlow 交换机，但不影响流表的内容和数据转发行为。

IETF 的标准化：

IETF 是全球互联网最具权威的技术标准化组织，负责互联网相关技术规范的研发与制定，网络设备厂商主导，更关注 SDN 相关功能和技术如何在网络中实现的细节上。

IETF 相关研究工作组：

转发与控制分组（ForCES）：

需求、框架、协议、转发单元模型、MIB 等，已发布 9 个 RFC 文档。

应用层流量优化工作组（ALTO）：

为应用层提供更多的网络信息，完成应用层的流量优化。

路由领域公开的研究组（IRSWG）：

研究 I2RS 的问题描述、需求、应用场景与架构模型等，在 RFC7921 中定义了 I2RS 支持的 SDN 系统架构。

ITU-T 的标准化：

ITU-T 提出了 SDN 的实现架构。研究的 SDN 的场景对象、相关架构主要针对运营商网络的。Y.FNsdn 项目，Y.FNsdn-fm 项目。

ETSI 的标准化：

成立网络功能虚拟化标准工作组（NFVISG），制定支持虚拟功能硬件和软件基础设施的要求和架构规范以及发展网络功能的指南。

3.SDN 体系结构:

数据平面与控制平面之间利用 SDN 控制数据平面接口(control-data-planeinterface, 简称 CDPI)进行通信, CDPI 具有统一的通信标准, 目前主要采用 OpenFlow 协议。

控制平面与应用平面之间由 SDN 北向接口(northbound interface, 简称 NBI)负责通信, NBI 允许用户按实际需求定制开发。

SDN 的体系架构:

(1) 北向接口:

应用平面与控制平面之间的接口 (NB), 向应用层提供抽象的网络视图, 使应用能直接控制网络的行为。

开放的、与产商无关的接口。

北向接口关键技术:

SDN 北向接口设计: 控制器将网络能力封装后开放接口, 供上层业务调用。

RESTAPI 成为 SDN 北向接口的主流设计。

交换机状态采集、静态流表推送、防火墙策略等多种类型的接口。

(2) 南向接口:

设备控制信令, 控制设备的转发行为。控制平面和数据平面之间的接口 (CDPI)。

功能: 转发行为控制、设备性能查询、统计报告、事件通知等。

ONF 体系架构: 标准化的南向接口协议 (Openflow), 不依赖于底层具体厂商的交换设备。

控制器/Controller: 对网络的抽象层, NOS 网络操作系统, 屏蔽硬件, 为应

用开发提供开发接口。

南向接口关键技术：

转发面开放协议：允许控制器控制交换机的配置以及相关转发行
为。

ONF 定义的转发面开放协议：Openflow 协议

传统网络面对灵活需求应接不暇的本质原因包括"只可配置,不可编程"以及“分布
式控制与管理架构带来的制约”。

SDN 的智能核心组件在 控制平面。

ETSI 重点关注 SDN 和 NFV 的技术融合，以实现虚拟化要求。

SDN 核心设计思想中,解耦是控制平面和 数据平面进行分离。

QoS 的设计属于网络生命周期的哪个阶段：规划与设计阶段。

首个基于 python 的 SDN 控制器是 POX。

第二章、 SDN 的基本原理

1. SDN 的定义：

表 2.1 狭义 SDN 和广义 SDN 主要区别

主 要 区 别	SDN 概念	
	狭义 SDN	广义 SDN
网络架构	ONF 基于 OpenFlow 三层架构	ONF 基于 OpenFlow 三层架构，IETF 基于 I2RS 的 SDN 架构，Overlay 架构
南向接口协议	OpenFlow	OpenFlow、NETCONF、OVSDB、OpFlex、PCEP、XMPP、I2RS 等
应用场景	软件定义数据中心	软件定义广域网、软件定义安全、软件定义接入、软件定义存储，软件定义一切

2. SDN 的体系架构：

SDN的主流技术架构

ONF定义的基于OpenFlow的架构	IETF提出的技术架构
特点：1、转发与控制分离； 2、标准化转发面。	特点：1、开放网络设备能力； 2、标准化API。
优点：易于流量调度	优点：1、充分利用现有设备； 2、快速实现。
NICIRA提出的Overlay技术架构	ETSI提出的NFV技术架构
特点：1、网络边缘软件化； 2、Overlay技术。	NFV与SDN技术互补； 但有本质区别，严格来说不能算是。
优点：1、与物理网络解耦； 2、部署灵活。	

3. SDN 设计的核心思想：网络能力抽象、网络可编程、数据平面与控制平面解耦

数据平面与控制平面的解耦：

通过解耦合，控制平面负责上层的控制决策，数据平面负责数据的交换转发，双方遵循一定的开放接口进行通信。

实现网络逻辑集中控制的前提。

网络功能的抽象：

ONF 网络架构实现转发抽象、分布状态抽象和配置抽象：

转发抽象 (forwarding abstraction)：隐藏了底层的硬件实现，转发行为与硬件无关；

分布状态抽象 (distribution abstraction)：屏蔽分布式控制的实现细节，为上层应用提供全局网络视图；

配置抽象 (specification abstraction)：网络行为的表达通过网络编程语言实现，将抽象配置映射为物理配置。

Overlay 网络架构实现对基础网络设施的抽象。

网络可编程：

网络可编程接口：

北向接口：REST (Representational State Transfer) API, RESTCONF 协议

南向接口：OpenFlow、OF-Config、NETCONF、OVSDB、XMPP、PCEP、

I2RS、OPFlex 等协议；

东西向接口：负责控制器之间的通信，未形成统一标准。

数据平面可编程技术：Intel 主导的 DPDK、斯坦福大学主导的

P4(Programming Protocol-independent Packet Processors)等。

4.题目

属于狭义SDN南向接口协议的是？

- ☐ (A) OVSDB
- ☒ (B) OpenFlow
- ☐ (C) NETCONF
- ☐ (D) PCEP

狭义 SDN（即 ONF 定义的 SDN 架构）中，OpenFlow 是最核心的南向接口协议，它专门用于实现控制器与数据平面设备（如交换机）之间的通信，负责下发流表、控制数据转发行为。

- A. OVSDB：是 Open vSwitch 的数据库管理协议，主要用于配置和管理 OVS 交换机，不属于控制转发的南向协议。
- C. NETCONF：是网络配置协议，用于设备的配置管理，属于偏北向的配置类协议。

- **D. PCEP**: 是路径计算单元通信协议, 用于网络路径的计算与协商, 与 SDN 数据平面的转发控制无关。

基于 I2RS 的 SDN 体系结构的特点不包括?

- ☒ **A** 在网络设备和应用层之间增加编排器
- ☐ **B** 通过增加插件实现能力开放
- ☐ **C** 保留和重用现有协议
- ☒ **D** 直接实现能力开放

基于 I2RS (Interface to the Routing System, 路由系统接口) 的 SDN 体系结构, 核心是在现有网络基础上实现能力开放, 其主要特点包括:

1. **引入编排器** (选项 A): 在网络设备与应用层之间增加编排器, 负责协调设备能力与应用需求, 是 I2RS 架构的典型设计。
2. **插件化能力开放** (选项 B): 通过在现有网络设备上增加插件来实现能力开放, 而非直接改造设备, 这是 I2RS 的关键实现方式。
3. **兼容现有协议** (选项 C): 保留和重用 BGP、OSPF 等现有网络协议, 避免对现有网络的颠覆性改造, 是 I2RS 的重要设计原则。

而选项 D “直接实现能力开放” 不符合 I2RS 的设计思路 —— I2RS 是通过插件、编排器间接开放能力, 而非让设备直接对外暴露能力, 因此这不是它的特点。

在 SDN 体系结构中, 管理平面的主要功能是**负责一些静态的配置工作**。

简单网络管理协议中实现接口统一的是 MIB 库。

在 SDN 系统中有三种重要的网络能力与行为抽象, 分别是**转发抽象、分布抽象、配置抽象**。

命令提示符是基于 ASCII 的系统, 实际上是一个有效的语法分析器。

东西向接口是分布式控制器之间的通信接口。

SDN 体系结构是基本框架和 SDN 运行机理以及功能实现逻辑的融合。

SDN 控制器的典型组成包括:

- NBI (北向接口) 代理: 负责与上层应用交互, 提供北向接口。
- SDN 控制逻辑: 核心的决策与控制功能。
- SBI (南向接口) 驱动: 负责与底层网络设备交互 (如 OpenFlow 驱动), 而非题干中的 ACPI 驱动。

SDN 的编程接口按方向和作用主要分为三类:

- 北向接口 (NBI): 控制器向上层应用提供的接口, 用于应用对网络的控制与管理。
- 南向接口 (SBI): 控制器向下层网络设备提供的接口, 用于下发流表和获取设备状态 (如 OpenFlow 协议)。
- 东西向接口: 多控制器之间的通信接口, 用于实现控制器的协同与状态同步, 是构建分布式 SDN 控制平面的关键。

第三章、 SDN 数据平面

1. 数据平面概述

单纯从数据平面的数据传输与转发功能看, SDN与传统网络的数据有高度的相似性, 均具有数据包解析、转发策略匹配、包转发调度等功能且两者的处理流程也基本一致。但是, SDN通过将数据平面与控制平面解耦并提供可编程功能, 使SDN在数据平面的存在形态、运行机理和特性等方面有了很大的变化。

- **存在形态:** SDN数据平面独立存在于基础设施层设备中。
- **运行机理:** SDN数据平面中所有转发处理模块 (包括数据包解析、转发策略匹配和转发调度等) 都是与协议无关和通用可编程的。
- **运行特性:** SDN数据平面对上层应用与业务具有很高的可扩展性与灵活性。

2. OpenFlow 交换机模型

各版本主要特征:

版 本 号	发 布 时 间	主 要 特 性
v1.0	2009.12	单流表, 只支持 IPv4
v1.1	2011.02	支持多级流表、组表、MPLS 和 VLAN 标签
v1.2	2011.12	支持可扩展匹配、多控制器、IPv6 基本头部
v1.3	2012.04	支持计量表、IPv6 扩展头部
v1.4	2013.10	支持流表同步机制、光端口
v1.5	2014.12	支持出向流表

OpenFlow 交换机模型架构:

在该架构中, 至少包括3个组成部分。

(1)流表(flow table): 由若干流表项(flow entry)构成, 每个动作(action)关联一个流表项, 指示交换机如何进行流的处理。

(2)安全通道(secure channel): OpenFlow交换机通过安全通道与远端控制器连接, 负责控制器与交换机之间的交互。

(3)OpenFlow协议: 定义了一种南向接口标准, 为控制器与交换机之间的通信提供了一种开放标准。

流表:

1) 包头域:

包头域是用于和交换机接收到的数据包进行匹配的元组, 包含 12 个字段, 称为 12 元组(12-tuple), 这些字段涵盖了 OSI 网络模型中第 2~4 层的网络特征。每一个元组中的数值可以是一个确定的值或者是 ANY, ANY 表示支持匹配任意值。

2) 计数器:

计数器主要对交换机中的每张流表、每个数据流、每个设备端口及每个转发队列进行维护, 用于统计数据流量的相关信息。

3) 动作表:

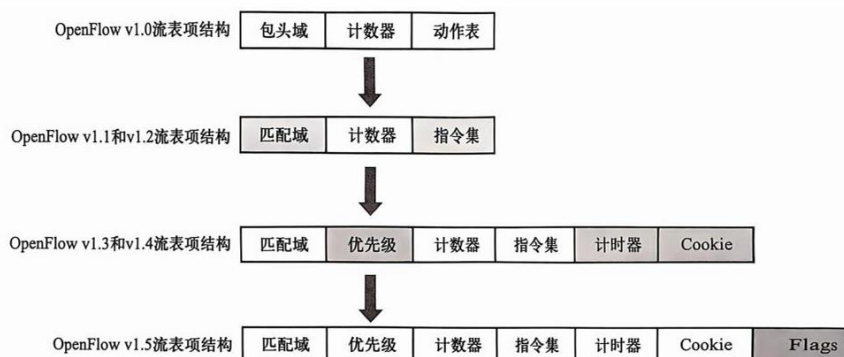
OpenFlow 流表项的动作表用于指示交换机如何对接收到的匹配数据包进行处理。流表项的动作分为必备动作(required action)和可选动作(optional action)两种类型。其中, 必备动作是所有 OpenFlow 交换机默认支持的, 而可选动作是交换机所能支持的可选动作种类 (由交换机告知控制器)。

表 3.3 OpenFlow 流表动作列表

动作类型	具体动作	动作说明
必备动作	转发 (forward)	转发数据包到物理端口或以下几类虚拟端口: LOCAL、ALL、CONTROLLER、IN_PORT、TABLE
	丢弃 (drop)	丢弃所有匹配的数据包
可选动作	转发 (forward)	转发数据包到两类虚拟端口: NORMAL、FLOOD
	入队 (enqueue)	转发数据包到端口绑定的队列, 提供基本的 QoS 支持
	修改包头 (modify-field)	修改数据包的包头, 可以是 12 元组中的任意字段

2. OpenFlow 流表项的结构演变

随着OpenFlow协议的版本迭代, 流表项的结构随之有所更新与变化。各版本流表项结构的变化如图3.6所示。



流表匹配:

1. 单一流表匹配处理

在OpenFlow v1.0中, 每个OpenFlow交换机只有一张流表。单一流表匹配相对简单, 相当于单一数据库表的查找与匹配过程。如图3.9所示, 在数据包进入OpenFlow交换机后, 首先进行包头的解析, 然后进入匹配阶段。

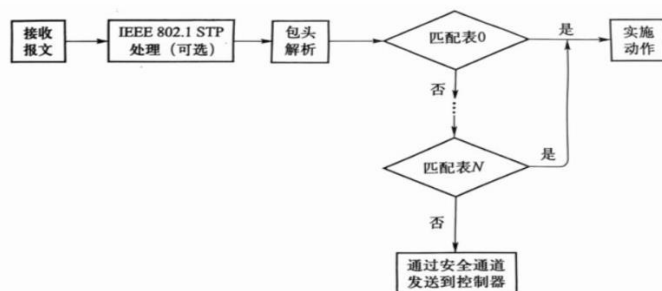


图 3.9 数据包匹配处理流程图

2. 多流表匹配处理

OpenFlow v1.0的单流表处理模式简单、易实现，但在网络需求变得越来越复杂时，各种策略放在同一张流表中就显得十分臃肿，而且影响查找与匹配效率。因此，从OpenFlow v1.1开始，提出了多流表(multiple flow table)架构。在多流表架构中，数据包在交换机中的匹配是以流水线的方式处理的，如图3.11所示，并在后续版本中沿用了这一处理方式。

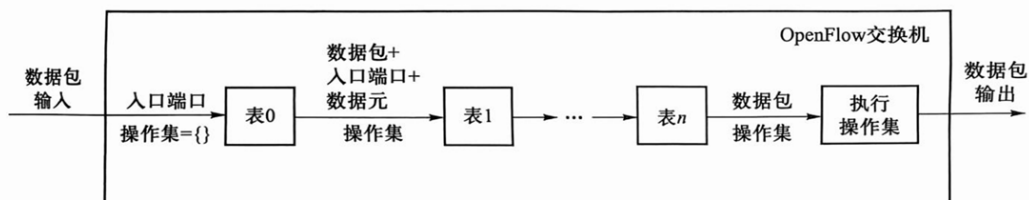


图 3.11 多流表匹配的流水线处理过程

3. SDN 交换机：

常用交换芯片：

芯片类型	优 点	缺 点	应 用 场 景
通用 CPU	易扩展、通用性强	处理性能较低	应用于网络设备的控制和管理
ASIC	性能高、处理能力很强	不易扩展、研发周期长	应用于实现各种成熟的协议
FPGA	支持反复擦写、可编程	处理能力有限	应用于科研和验证
NP	可编程、可进行复杂的多业务扩展	性能不及 ASIC	应用于路由器、防火墙等协议更为复杂的网络设备

物理交换机：

物理交换机又称硬件交换机。根据交换芯片的类型，SDN 物理交换机根据采用的芯片不同主要分为以下三类。

1) 基于 ASIC 的 SDN 交换机

又可细分为基于 ASIC 的 SDN 品牌交换机和基于 ASIC 的 SDN 白盒交换机。

2) 基于 NP 的 SDN 交换机

网络处理器 (NP) 是一种可编程的处理器，利用众多并行运转的微码处理器进行复杂的多业务扩展，适于实现创新或未成熟的业务。但性能和 ASIC 相比仍存在差距，

多用于路由器、防火墙等协议中更为复杂灵活的网络设备。

3) 基于 FPGA 的 SDN 交换机

FPGA 是一种门阵列芯片，它支持反复擦写，可以通过编程改变电路结构，以实现不同的网络功能，但其处理能力有限，难以实现大规模的网络转发，因此 FPGA 主要应用于科研和验证。

虚拟交换机：

虚拟交换机由于硬件交换机的研发投入较高，而虚拟交换机通过软件实现，因此成本更低、配置更为灵活，虽然性能与硬件交换机有一定差距，但基本能满足中小网络的要求。虚拟交换机又称软件交换机。目前已经实现的开源虚拟交换机主要有 Open vSwitch (OVS)、Indigo、Ofsoftswitch13、LINC 等。

4. 通用可编程数据平面：

“协议无关交换机架构” (protocol-independent switch architecture, PISA)。

PISA 是一种具有完全可编程数据包解析器和通用“匹配-动作”单元的高速数据包处理器架构。

其中，可编程解析器提供了协议无关的数据包协议解析处理功能，改进了 OpenFlow 交换机在支持新网络协议方面的不足；PISA 中包含了大量通用“匹配-动作”单元，每个单元包括“匹配”逻辑和“行动”逻辑，这些“匹配-动作”逻辑是通过静态随机存储器(SRAM)、三态内容寻址存储器(TCAM)、寄存器、通用哈希表和支持通用数据包处理指令集的计算逻辑单元来实现的。

与 OpenFlow 交换机模型相比，解析器和“匹配-动作”单元只有在被编程后才能做具体的数据包处理工作，PISA 芯片在没有配置前不会实现任何数据平面协议，因此 PISA 是协议无关的架构。

除了可编程解析器和通用的“匹配-动作”流水线。为了提高可编程性，PISA 还提供了一条回流路径使一些特殊数据包能够被多次反馈到解析器和流水线上。编译器也可以使用这个回流功能来实现传统芯片因只有固定的查找修改单元而无法完成的复杂逻辑。

数据平面编程语言：

要对 PISA 或其他类型的可编程数据包处理器进行编程，就需要一种专用的数据平面编程语言。P4 就是一种协议无关的数据包处理编程语言，它支持用户自定义匹配字段、协议解析过程和转发过程，从而能实现真正意义上的协议无关可编程数据平面。

P4 的抽象转发模型：

表 3.6 V1Model 各模块作用	
模 块	作 用
解析器（parser）	根据预先定义的报文头部格式对数据包进行解析，并提取头部的各个字段。在解析器中根据报文头部的协议字段匹配下一步需要解析的协议头部。例如，若以太网帧头中的协议字段是 0x0800，则下一步需要解析的是 IP 分组头部；如果是 0x0806，则下一步需要解析的是 ARP 报文，会进行不同的处理
流水线入口（ingress）	流水线部分，数据处理逻辑的入口
续表	
模 块	作 用
流管理（traffic manager, TM）	定义核心的处理逻辑，并且对一些元数据进行更新
流水线出口（egress）	流水线部分，数据处理逻辑的出口
重组（deparser）	用于对数据包进行重组，因为数据包在处理过程中经历了分解和处理，所以最后转发的时候需要重组数据包

P4 程序定义：包头、解析器、表、动作、控制程序。

4. 题目：

包头域字段通常仅对应 OSI 模型的 **2-4 层**：

- **第 2 层（数据链路层）**：如以太网帧头，包含 MAC 地址等信息。
 - **第 3 层（网络层）**：如 IP 包头，包含 IP 地址、TTL 等信息。
 - **第 4 层（传输层）**：如 TCP/UDP 头，包含端口号、校验和等信息。
- 而第 5 层（会话层）及以上（表示层、应用层）的信息属于数据包的载荷部分，并不在包头域中。

下列哪个匹配字段不是数据链路层的标识

- ☐ A mac地址
- ☒ B 输入端口
- ☐ C 以太网类型
- ☐ D vlan优先级

- **A. MAC 地址**：是数据链路层的核心标识，用于在局域网中唯一标识设备，属于数据链路层。
- **B. 输入端口**：是设备的物理 / 逻辑端口标识，属于设备的接口属性，而非数据链路层协议包头中的字段，因此不是数据链路层的标识。
- **C. 以太网类型**：是以太网帧头中的字段，用于标识上层协议（如 IP、ARP），属于数据链路层。
- **D. VLAN 优先级**：是 VLAN 标签（802.1Q）中的字段，用于标识数据帧的优先级，属于数据链路层。

哪个不是必备动作转发的端口

- ☒ A TABLE
- ☐ B FLOOD
- ☐ C LOCAL
- ☐ D ALL

- **A. TABLE**：是必备的保留端口，用于让数据包再次进入流表进行匹配，实现复杂的转发逻辑。
- **B. FLOOD**：是泛洪动作，仅用于未知单播、广播等场景，属于**可选动作**，不是必备的转发动作端口。
- **C. LOCAL**：是必备的保留端口，用于将数据包发送到本地控制器，支持控制器对数据包的决策处理。
- **D. ALL**：是保留端口，用于将数据包发送到除输入端口外的所有端口（包括 STP 阻塞端口），在特定广播场景中是必备的。

哪个不是V1.3和V1.4的流表项结构

- ☐ A 匹配域
- ☒ B Flags
- ☐ C 指令集
- ☐ D Cookie

我们来分析 OpenFlow V1.3/V1.4 的流表项核心结构：

- **A. 匹配域：**是流表项的核心部分，用于定义数据包的匹配规则（如 MAC 地址、IP 地址、端口等），属于 V1.3/V1.4 的必备结构。
- **B. Flags：**是 OpenFlow V1.0 版本的流表项字段，在 V1.3 及之后的版本中已被移除，不属于 V1.3/V1.4 的流表项结构。
- **C. 指令集：**V1.3 及之后版本用指令集（Instructions）替代了旧版本的动作集，用于定义匹配后的处理逻辑（如执行动作、跳转至其他流表等），属于必备结构。
- **D. Cookie：**是流表项的唯一标识字段，用于控制器跟踪和管理流表项，属于 V1.3/V1.4 的必备结构。

openflowv1.0 只支持 ipv4

传统数据包的转发策略主要查询什么表来完成

- ☐ A BGP
- ☐ B OSPF
- ☒ C FIB
- ☐ D RIB

- **C. FIB（转发信息库）：**是路由器 / 交换机的转发表，存储了数据包转发所需的下一跳、出端口等信息，传统数据包转发时正是通过查询 FIB 来完成转发决策。
- **D. RIB（路由信息库）：**存储了路由协议学习到的所有路由信息，用于计算生成 FIB，并不直接参与转发查询。

流表匹配中匹配到的表项中未定义具体动作,下一步是

- ☐ A 批量转发
- ☒ B 继续匹配
- ☐ C 丢弃数据包
- ☐ D 申请新的表项

在 OpenFlow 协议的流表匹配规则中：

- 当数据包匹配到流表项，但该表项未定义具体动作时，设备会执行**默认丢弃动作**。这是 OpenFlow 的安全机制，避免无明确转发规则的数据包被随意转发，引发网络风险。
- 选项 B “继续匹配” 是当数据包**未匹配到任何流表项**时的行为（如跳转至下一个流表），而题目中是“匹配到表项但无动作”，因此不适用。
- 选项 A “批量转发” 和 D “申请新的表项” 均不符合 OpenFlow 的协议逻辑，设备不会主动申请表项，也不会无动作时批量转发。

FIB是根据什么表创建的

- ☐ A BGP
- ☐ B AAA
- ☐ C OSPF
- ☒ D RIB

RIB（路由信息库）：存储了所有路由协议（如 BGP、OSPF）学习到的路由信息，路由器会从 RIB 中选择最优路由，进而生成 **FIB（转发信息库）**，因此 FIB 是根据 RIB 创建的。

流表匹配中,优先级最高的是表项 0。

下列哪个选项不属于openflow v1.0交换机模型的组成部分

- ☐ A openflow协议
- ☒ B 组表
- ☐ C 流表
- ☐ D 安全通道

- **A. OpenFlow 协议**：是控制器与交换机之间的通信协议，属于 v1.0 模型的核心组成。
- **B. 组表**：是在 **OpenFlow v1.1** 及之后版本才引入的特性，用于实现多端口转发、负载均衡等复杂动作，**不属于 v1.0 的交换机模型组成**。
- **C. 流表**：是 v1.0 模型的核心组件，存储匹配规则和对应的动作，负责数据包的转发决策。
- **D. 安全通道**：是控制器与交换机之间的加密通信通道，用于传输 OpenFlow 消息，属于 v1.0 模型的必备组成。

匹配字段不在采用固定长度是openflow哪个版本的特性

- ☒ A v1.2
- ☐ B v1.3
- ☐ C v1.0
- ☐ D v1.1

包头域包含多少个字段

- ☐ A 9
- ☐ B 15
- ☒ C 11
- ☐ D 12

哪个不是组表的表项

- ☐ A 组表号
- ☐ B 计数器
- ☒ C 计量带
- ☐ D 动作桶

第四章、南向接口协议

1. 概述：

第一类用于实现数据平面与控制平面的信息交互。向上收集交换机信息，比如交换机的特性、配置信息、工作状态等；向下下发控制策略，指导数据平面设备的转发行为。OpenFlow、XMPP、I2RS 和 OpFlex 都属于此类协议。

第二类用于实现网络的配置与管理。也就是说，这类协议专注于控制器与交换机之间有关设备配置与管理信息交互的实现。OF-Config。

第三类主要面向流量工程(traffic engineering, TE)的实现。流量工程是指根据各种数据业务流量的特性选取传输路径的处理过程,用于平衡网络中的不同交换机、路由器以及链路之间的负载。支持流量工程的实现,需要收集基础设施层的拓扑与链路、流量与负载等信息。以实现路由计算为例,需要收集底层链路的拓扑、带宽、开销和连通状态等信息。PCEP 就属于此类协议。

表 4.1 南向接口协议的目标与功能定位		
协议名称	制定或发布者	设计目标
OpenFlow	ONF	用于 OpenFlow 交换机与控制器之间的信息交互，提供双方通信的标准，适于支持 OpenFlow 交换机与控制器之间的通信
OF-Config	ONF	用于 OpenFlow 交换机的配置与管理，提供控制器连接设置、加密、队列、端口等配置能力，支持双向认证、可靠传输、事务管理等服务，适合作为 OpenFlow 的伴侣协议
NETCONF	IETF	用于网络设备的配置与管理，提供获取、添加、修改或删除网络设备的配置数据等功能，适于对非 OpenFlow 设备的配置与管理
OVSDB	VMware/IETF	用于 Open vSwitch 的配置与管理，提供对设备数据库的增、删、改、查等操作，适于对 Open vSwitch 交换机的配置与管理
XMPP	IETF	用于即时通信的协议，具有良好的可扩展性，对现有网络设备的兼容性较好，适合作为 OpenFlow 的一种替代或者补充协议
PCEP	IETF	用于流量工程中路径计算的通信协议，可以与 OpenFlow 结合使用，在流量工程的应用场景中实现传统网络和 SDN 网络的统一管理和调度
I2RS	IETF	用于控制器与非 OpenFlow 设备之间的通信协议，适于 IETF 定义的 SDN 体系结构中的南向接口协议
OpFlex	思科	用于思科 ACI 体系中的策略控制协议，提供了双向的通信能力，适于支持 OpFlex 协议的控制器与设备之间的通信

2. OpenFlow 协议

消息类型：

1. Controller-to-Switch 消息

Controller-to-Switch 消息由控制器发起，发送给交换机。其主要用于管理和获取交换机状态，并可能会要求交换机回复对应的消息。此消息的功能特性可进一步细分为以下几类。

(1)Features：用于获取交换机的特性或属性信息。在建立传输安全会话时，控制器主动向交换机发送 Features-request 消息，请求交换机的特性信息，交换机通过 Features-reply 消息回复自身所支持的功能特性。

(2)Configuration：用于设置或查询交换机的配置信息，若为设置配置信息，交换机只需执行信息；若为查询配置信息请求，交换机需给予应答。

(3)Modify-state：用于修改交换机的状态消息，如用于增加、删除和修改交换机的流表项、组表项及端口状态。

(4)Read-state: 用于读取与收集交换机的各种状态信息, 包括配置信息、统计信息和功能特性。

(5)Send-packet: 用于向交换机的指定端口发送数据包, 在 OpenFlow v1.1 以后的版本中, 该类型改成 Packet-out 消息, 用于响应 Packet-in 消息的处理。Packet-out 消息必须携带一个 Buffer_id 来定位缓存在交换机上的数据包, 此外, 该消息还需携带用于指导数据处理的动作集, 如果动作集为空, 丢弃数据包。

(6)Barrier: 用于保证消息执行的顺序。Barrier 消息会在所有的前置消息完成后才处理。

2.Asynchronous 消息

Asynchronous 消息通过异步方式由交换机上传给控制器, 用于传递网络事件和交换机状态的变化更新, 如告知控制器新数据包的到达和交换机状态的改变等。其细分的功能特性如下。

(1)Packet-in: 如果数据包在流表中没有匹配项, 或者匹配的流表项中有指定端口为 CONTROLLER 的动作时, 交换机将发送 Packet-in 消息给控制器。如果交换机有足够的缓存用于存储发往控制器的数据包, 则 Packet-in 消息只需要将包头中的控制信息 (默认 128B) 和交换机缓存标识 (Buffer_id) 发送给控制器, 控制器在 Packet-out 消息中可以根据 Bufferid 处理交换机缓存中对应的数据包; 如果交换机不支持缓存 (或者缓存容量不足), 则将整个数据包封装进 Packet-in 消息发给控制器。

(2)Flow-removed: 用于交换机向控制器主动发送流表被移除的信息。当交换机中的流表项因超时或修改等原因被删除时, 会触发该消息的发送。

(3)Port-status：用于交换机向控制器主动发送端口状态变化的信息。

当交换机的端口状态发生变化时(Up/Down)，会触发该消息的发送。

(4>Error：用于交换机向控制器主动发送运行错误的信息。例如，当交换机产生运行时错误时，会通过消息包主动告知控制器所发生的错误信息。

3.Symmetric 消息

Symmetric 消息可以由控制器或交换机任意一方发送，而无须对方的许可或请求。其细分的功能特性如下。

(1>Hello：用于控制器和交换机双方建立安全连接时，交换连接握手消息。(2)Echo：用于测量链路延迟与带宽，或用于保持活动连接。双方均可主动发送 Echo-request 消息给对方，接收者需要回复 Echo-reply 消息。

(3)Vendor:用于厂商自定义的消息扩展，以为交换机生产厂商提供额外的定制功能预留空间。

OpenFlow 协议数据包由 OpenFlow 消息头(OpenFlow header)和 OpenFlow 消息体(OpenFlow message)两部分组成。其中,OpenFlow 消息的头部结构是固定的，包含 4 个字段(64b)数据结构。其中，Version 代表协议的版本号；Type 代表消息类型，消息类型是一个以 OFPT_为前缀的枚举类型常量，详见 OpenFlow 规范文件；Length 代表数据包的长度；Xid 代表数据包的 Transaction_ID，Reply 消息会使用 和 Request 消息中相同的 Transaction_ID。

Version (8)	Type (8)	Length (16)
Xid (32)		

图 4.2 OpenFlow 消息头格式

3. Hello 消息

Hello 消息(OFPT_HELLO)用于控制器与交换机在建立连接时协商协议版本,该消息只包含 OpenFlow 消息头,如图 4.3 所示,Version 表示发送方所支持的 OpenFlow 协议的最高版本号,双方选取 Hello 消息中最低版本的协议作为通信协议。通过在字段 Type 中指定该消息的类型为 OFPT_HELLO 来表明其消息类型。

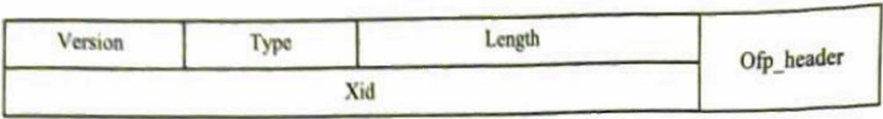


图 4.3 OFPT_HELLO 消息格式

4.Features 消息

该消息包括 Features-request 和 Features-reply 两种不同的格式。其中,Features-request 在 OpenFlow 连接建立后,由控制器向交换机发送,请求获取交换机的特性信息,包括交换机的数据通路 ID、缓冲区数量、端口及端口属性等。Features-request 消息只包含 OpenFlow 消息头,通过在字段 Type 中指定该消息的类型为 OFPT_FEATURES_REQUEST 来表明其消息类型。

5.Configuration 消息

该消息包括 Set-config、Get-config-request 和 Get-config-reply 三种格式,分别通过在 OpenFlow 消息头字段 Type 中指定相应的消息类型来表示,分别为 OFPT_SET_CONFIG、OFPT_GET_CONFIG_REQUEST 和 OFPT_GET_CONFIG_REPLY。控制器通过 Set-config 消息(OFPT_SET_CONFIG)对交换机进行配置,通过 Get-config-request 消息(OFPT_GET_CONFIG_REQUEST)获取交换机的配置参数,交换机则通过 Get-

config-reply 消息(OFPT_CET_CONFIG_REPLY)对控制器发出的 Get-config-request 消息进行回应。OFPT_GET_CONFIG_REQUEST 只包括 OpenFlow 消息头。OFPT_SET_CONFIG 和 OFPT_GET_CONFIG_REPLY_消息除了 OpenFlow 消息头, 还包括 Flags 和 Miss_send_en 两个字段, 如图 4.5 所示。Flags 用于指示交换机处理 IP 分片数据包, Miss_send_en 用于指示当交换机无法处理的数据包到达时, 将数据包发给控制器的最大字节数。

6.Packet-in 消息

该消息被交换机用于向控制器发送包接收的信息。但只有当数据包在交换机的流表中没有匹配项, 或者匹配的流表项中所指定端口为 CONTROLLER 的动作时(Output=CONTROLLER), 交换机才发送 Packet-in 消息(OFPT_PACKETIN)给控制器, 以提示控制器进行相应的转发决策处理。

7.Modify-state 消息

控制器利用 Modify-state 消息(OFPT_FLOW_MOD)对交换机的流表进行配置, 包括增、删、改等操作。

3.NETCONF 协议

1.安全传输层

安全传输层用于为 NETCONF 连接提供安全、可靠的传输功能, 涉及序列号、认

证、数据完整性和安全保障等机制。安全传输层承载在 SSH、TLS 等协议基础上,

也就是说可以选择 SSH、TLS 等现有传输安全协议建立安全连接。当一

个安全连接

建立后，客户端与服务器会通过 Hello 消息进行一次 NETCONF 消息交互，用于确认双方支持的能力。

2.消息层

消息层用于为编码 RPC 与通知(notification)提供一个简单、独立于传输层协议使用

<pc>的框架机制。借用 RPC 协议的通信机制，RPC 是一种通过网络从远程计算机

程序上请求服务，而不需要了解底层网络技术的协议。NETCONF 和 <rpc-reply>元

素来提供独立于承载协议的 NETCONF 请求和响应，，完成对设备的配置和管理，

在交换 Hello。消息后，NETCONF 客户端向服务器端发送<rpc>消息对服务器端设

备进行配置管理，服务器端对每个<rpc>请求发送<rpc-reply>进行响应。

3.操作层

操作层是 NETCONF 协议的核心，用于为协议提供基于 RPC 的操作集。该层定义了一个底层操作的集合用以配置和获取设备信息，而设备信息一般存放在数据库中，所以定义的操作提供了对数据库信息的获取、配置、复制和删除等功能。协议提供的基本操作如表 4.2 所示。根据命令名称就可大概知道它的作用，例如，<get-config>用来从<running/>、<candidate/>和<startup/>配置数据库中获

取全部或指定的一部分配置数据，<edit-config>用来修改、创建或删除配置数据。

4. 内容层

内容层提供了关于网络管理所涉及配置数据的描述，使用 YANG（一种数据建模语言，详见 RFC6020）进行建模。

4. 题目：

OVSDB 管理协议使用 JSON 作为数据格式的描述语言

Configuration 消息包括 **Set-config**、**Get-config-request** 和 **Get-config-reply** 三种格式

在 OVSDB 的 JSON 模式定义中，列模式（column-schema）是描述表中列的结构 JSON 对象，它包含以下核心字段：

type：必填字段，定义列的数据类型。

ephemeral：可选字段，标记该列是否为临时（会话结束后自动删除）。

mutable：可选字段，标记该列的值是否可修改。

Echo 消息的功能特性是**用于测量链路延迟与带宽**

OpenFlow 消息头中，消息类型是一个以 OFPT_为前缀的枚举类型常量

不属于OpenFlow v1.3 伴侣协议的是？

☒ A OF-Config v1.0

☐ B OF-Config v1.2

☐ C OF-Config v1.1

XMPP 属于第几类南向接口协议？ 第一类。

Asynchronous 消息通过异步方式由交换机上传给控制器，用于传递网络事件和交换机状态的变化更新。

不属于NETCONF 协议模型分层架构的是

- ☐ (A) 安全传输层
- ☒ (B) 基础设施层
- ☐ (C) 内容层
- ☐ (D) 消息层

下列说法错误的是

- ☐ (A) 一个控制器可控制多个OpenFlow 逻辑交换机
- ☒ (B) 逻辑交换机包含使能交换机
- ☐ (C) OF-Config协议在设计实现中，对交换机进行了功能组件的逻辑抽象
- ☒ (D) 一个OpenFlow 逻辑交换机可被多个控制器所控制

下列哪一个不属于controller-to-switch消息

- ☐ (A) features
- ☐ (B) Send-packet
- ☐ (C) Configuration
- ☒ (D) Error

在 OpenFlow 协议中，消息分为三大类：**Controller-to-Switch**（控制器主动发给交换机）、**Asynchronous**（交换机主动发给控制器）、**Symmetric**（双向均可发起）。我们逐一分析选项：

- **A. features:** Features Request 是控制器主动向交换机发送的查询请求，属于 Controller-to-Switch 消息。
- **B. Send-packet:** 即 Packet-Out 消息，是控制器命令交换机发送数据包，属于 Controller-to-Switch 消息。
- **C. Configuration:** Configure 消息是控制器配置交换机参数（如 miss_send_len），属于 Controller-to-Switch 消息。
- **D. Error:** Error 消息是交换机主动向控制器报告错误事件，属于 ****Asynchronous（异步）**** 消息，不属于 Controller-to-Switch（控制器主动发起）的范畴。

OpenFlow 消息的头部结构是固定的，由 4 个字段组成

一个 I2RS 底层设备可以与多个 I2RS 控制器对接，一个 I2RS 控制器可控制多个 I2RS 底层设备

不属于netconf操作层的是

- ☐ A kill-session
- ☒ B running
- ☐ C get
- ☐ D lock

NETCONF 协议的架构分为内容层、操作层、RPC 层和传输层，我们来分析每个选项：

- **A. kill-session:** 属于操作层操作，用于主动终止一个 NETCONF 会话。
- **B. running:** 这是 NETCONF 定义的**配置数据存储（datastore）类型**（还有 startup、candidate 等），属于内容层的概念，不是操作层的操作。
- **C. get:** 属于操作层核心操作，用于从设备获取配置或状态数据。
- **D. lock:** 属于操作层操作，用于锁定配置存储，防止并发修改。

netconf 中,close-session 是用来强制关闭某一个会话？

这个说法是**错误的** ✕

在 NETCONF 协议中：

- **close-session** 用于主动、优雅地关闭当前会话，由会话的发起方（如客户端）主动发起，结束自己的连接。
- **kill-session** 才是用于**强制关闭指定的某一个会话**，通常由服务器 / 控制器发起，用来终止异常或超时的其他会话。

第五章 SDN 控制平面

1.概述及其架构

在 SDN 架构中，控制器是核心，它是连接底层交互设备与上层应用之间的桥梁。一方面，控制器通过南向接口协议实现对底层网络交换设备的集中管理与状态监测，为数据平面提供转发决策和流量调度；另一方面，控制器通过北向接口向上层应用开放多个层次的可编程能力，允许用户根据特定的网络应用场景灵活地定制相应的运行与管理策略，并为开发人员提供灵活的开发接口，为用户提供方便使用的操作接口。

(1)协议适配层：主要实现对底层多种协议的适配，包括支持多种南向接口协议。

(2)基础网络层：这是该体系模型中必不可少的核心层，用于实现关键的网络控制逻辑，包含控制器内部的实现逻辑和一些底层的网络实现逻辑。

(3)内置应用层：用于提供一些常用的基础功能包。

(4)北向接口层：用于提供与应用层交互的北向接口。

(5)配置管理层：该层实现包括软件管理、集群管理和用户界面(user interface, UI)管理等功能。

其中前 4 个为控制功能部分,最后面那个为管理功能部分.

SDN 控制器基础功能:

1.链路发现

链路发现是指控制器利用南向接口的上行通道，收集底层交换设备上报的链路状态相关信息，并经过统计和分析获得对底层链路的发现及其状态信息。

目前，在 SDN 链路发现的技术实现上，主要采用链路层发现协议(LLDP)，该协议定义在 IEEE 802.1ab 中，是一个二层协议，广泛应用于网络设备的发现，其中也包括 SDN 网络。

2. 拓扑管理

拓扑管理为了随时监控和采集网络中 SDN 交换机的信息，及时反馈网络的设备工作状态和链路链接状态，以形成对网络拓扑及其变化的准确认识。拓扑管理还有一项主要工作：在随时更新 SDN 交换机及链接状态的同时，对各种逻辑组网信息进行记录。

3.策略制定

南向网络控制中的核心功能之一是策略制定，交换流表生成算法是影响控制器智能化水平的关键因素，控制器需要针对不同网络层次的传输需求，制定相应的转发策略并生成对应的流表项。

2.开源控制器:

表 5.1 主要开源控制器列表		
名称	实现语言	特征简介
Mul	C	由 Kulcloud 公司开发，内核是一个基于 C 语言的多线程基础架构，用于托管应用的多层级北向接口
Trema	Ruby/C	由 NEC 开发，具有模块化的框架
NOX	C++	由 Nicira 开发，是业界第一款 OpenFlow 控制器，是众多 SDN 研发项目的基础
POX	Python	由 Nicira 开发，是 NOX 的纯 Python 实现版本，支持控制器原型功能的快速开发
Ryu	Python	由 NTT 开发，能够与 OpenStack 平台整合，具有丰富的控制器 API，支持网络管控应用的创建
Beacon	Java	由斯坦福大学开发，采用跨平台的模块化设计，支持基于事件和线程化的操作
Floodlight	Java	由 Big Switch Networks 开发，基于 Beacon，是企业级的 OpenFlow 控制器
OpenDaylight	Java	主要由设备厂商驱动，如思科、IBM、HP、NEC 等
ONOS	Java	由开放网络实验室 ON. Lab 推出

NOX 是 SDN 发展史上第一个控制器，由 Nicira 开发，并于 2008 年发布。

NOX 的底层使用 C++ 编写，支持 OpenFlowv1.0 协议。

POX 采用 Python 语言编写。

Ryu 基于 Python 语言开发，代码风格流畅，模块清晰，可扩展性很强。

3. 题目：

日本 NTT 开发的开源控制器是 Ryu

控制器体系架构中,实现关键控制逻辑的核心层是 基础网络层。

关于链路发现流程,不正确的是

- ☒ A 交换机不是OpenFlow交换机,则断开连接
- ☐ B 交换机接收到Packet-out消息,向除交换机和控制器相连端口之外的所有其他端口转发
- ☒ C 控制器向所有与之连接的交换机发送一个承载Packet-out消息的LLDP数据包
- ☐ D 交换机对所收到的LLDP数据包进行流表查找操作

• **选项 A：**“判断交换机是否为 OpenFlow 交换机并断开连接”是在 ** 初始握手阶段（Features Request/Reply）** 发生的动作，不属于“链路发现流程”的步骤。链路发现的前提是交换机已经和控制器成功建立 OpenFlow 连接，因此这个描述在“链路发现流程”的语境下是不正确的。

• **选项 B：**交换机收到携带 LLDP 的 Packet-out 消息后，向除与控制器相连端口外的所有端口转发，是链路发现中泛洪探测邻居的标准步骤，描述正确。

• **选项 C：**控制器向所有连接的交换机发送携带 LLDP 数据包的 Packet-out 消息，是链路发现的起始触发动作，描述正确。

• **选项 D：**交换机收到 LLDP 数据包时**不会**进行流表查找，而是直接上送给控制器。虽然这个描述本身是错误的，但题目要求选择“关于链路发现流程不正确的”，而 A 的错误在于“不属于链路发现流程的步骤”，因此 A 是符合题意的答案。

Floodlight支持的南向协议目前只有OpenFlow



基于 Beacon 开发的开源控制器是 Floodlight.

APIC 与 DNA-C 都是思科的商业控制器,一个面向数据中心,一个面向园区网。

控制器体系架构的控制功能部分,自下而上分为 4 层,分别为协议适配层、基础网络层、内置应用层和配置管理层

(错误,配置管理层不是控制功能部分的)

ODL 组织开发的开源控制器是 OpenDaylight

配置管理层主要实现软件管理、集群管理和用户界面管理等功能

Ryu 控制器,第一类和第二类南向接口协议都支持

LLDP 报文中,帧校验序列主要是校验数据帧,判断传输过程中是否出错或被篡改

关于SDN控制器扩展,描述不正确的是

- ☐ (A) 控制能力的集中化也意味着控制器的可靠性与安全性成为瓶颈
- ☐ (B) 控制器的软件化使服务器可以作为控制器软件的运行载体
- ☐ (C) 集中化控制对于把握全网资源视图、改善网络资源交付具有极其重要的作用
- ☒ (D) 多个SDN控制器通过标准的东向接口组成分布式集群,可以解决单点失效问题

关于SDN控制器的评估,描述不正确的是

- ☒ (A) 可扩展性只关注对应用与业务变化的弹性响应
- ☐ (B) 容错性或冗余备份是可靠性的重要体现
- ☐ (C) 可编程性包括控制器内部编程和外部编程
- ☐ (D) 控制器对于网络功能的支持是评估的首要因素

关于链路发现,说法不正确的是

- ☐ (A) 链路发现是指控制器利用南向接口的上行通道,收集底层交换设备上报的链路状态相关信息
- ☐ (B) SDN链路发现主要采用链路层发现协议
- ☐ (C) 链路发现可以形成完整的网络拓扑
- ☒ (D) 链路发现由交换机启动

不是SDN拓扑管理功能的是

- ☐ (A) 更新SDN交换机及连接状态
- ☒ (B) 生成交换流表算法
- ☐ (C) 监控和采集网络中SDN交换机的信息
- ☐ (D) 记录逻辑组网信息

第六章 SDN 北向接口

1.北向接口概述:

SDN 北向接口的功能主要体现在两方面，一是为上层业务应用提供网络资源与基础功能抽象，二是为上层业务应用和网络管控提供可编程的开发接口。借助北向接口所提供的这两方面功能，可通过网络抽象和虚拟化实现基本网络功能。

(1)功能型北向接口

此类接口基于**自下而上**的网络视角和方法，重点在网络资源抽象及控制能力的开放，关注的是**“网络能提供什么”**，可将底层网络的类型与特性抽象为不同类型的网络资源或逻辑功能，如拓扑、L2VPN、L3VPN、隧道等。

功能型北向接口主要面向具体的网络功能，与具体的网络实现技术方案相关。它主要对网络功能模型与网络管理模型进行抽象，网络功能模型主要在控制器上实现，侧重于业务下发与流量统计等功能；网络管理模型主要在运行支撑系统上实现，侧重于告警、性能监测和日志统计等功能。

(2)基于意图的北向接口

此类接口基于**自上而下**的网络视角和方法，重点在满足上层应用与业务实现逻辑和服务质量的需求关注的是**“应用需要什么”**，通过对用户或业务需求的意图分析与洞察，定义与抽象其所需要的连接服务、资源需求、访问控制、流处理和策略逻辑等内容。

北向接口的标准化:

1. ONF 的相关工作

1)CIM 工作组

CIM(通用信息模型)工作组致力于定义一个对于各种网络技术公共的信息模型。信息模型是对物理资源的抽象表示，它包含了一组对象、对象之间的关系、对象属性和对象可以执行的操作。

2)NBI 工作组

NBI（北向接口）工作组是 ONF 最早定义北向接口的工作组，主要研究功能型北向接口，重点是网络资源抽象及控制能力的开放。

2.IRTF 的工作

因特网研究任务部(IRTf)专门成立 SDNRG 研究组，主要研究 SDN 模型的分离、SDN 模型的可扩展性和适用性等,SDNRG 提出了 SDN 的层次化架构，在管理和控制平面上给出网络服务抽象层(NSAL)，也就是北向接口，为应用、服务、控制和管理提供接入服务，其他还包括应用平面、控制层分为控制抽象层和管理抽象层、设备和资源管理层。

3. IETF 的工作

因特网工程任务部(IETF)并没有独立的北向接口工作组，但是也成立了 SFC 工作组。SFC 的对外控制接口是 SDN 北向接口的重要组成部分。目前，SFC 工作组已经发布了 SFC 架构 RFC，并在努力制定相应的数据模型。

2. Rest API

REST API包含了“API”和“REST”两个关键词。其中，API通常包含了一组预先定义好的函数，供应用程序或编程开发人员访问调用。REST的英文全称是representational state transfer，直译成中文是“表述化的状态转移”，该词最早出现在2000年 Roy Thomas Fielding 的博士论文中。REST本质上是一组 Web架构的约束条件和原则，满足 REST约束条件和原则的设计规范或者架构风格就称之为RESTful，遵循RESTful 设计的API就是REST API。RESTful 并不是专门为SDN提出的，而是针对Web 应用中 HTTP 使用中的问题提出的。长期以来，在Web应用中，HTTP协议的使用很不规范、非常随意，甚至很混乱。例如，要删除一个ID号为10的学生信息，URL有多种写法，至少有下面三种比较普遍的写法：

```
/student/delete?id=10
/student?id=10&method=delete
/deleteStudentById?id=10
```

由此可见，URL的设计缺乏规范性，同时还存在一些其他的问题，比如HTTP 的动词使用不当、HTTP 的返回状态码使用不规范等。RESTful 正是要解决Web应用中的HTTP的诸如此类问题。如果用RESTful设计，上面的例子可以统一写成以下形式：

```
DELETE /students/10
```

这种以“一个HTTP动作+一个URI”的书写形式，显得非常规范和简洁。

1. Floodlight REST API

Floodlight 控制器的北向接口提供了丰富的 RESTAPI，除了模块管理等控制器内部的功能模块外，Floodlight 中的应用模块，如虚拟网络过滤器模块、静态流表项推送模块防火墙模块、学习交换机模块等都提供了 RESTAPI，可以供上层应用调用。

2. Ryu REST API

Ryu 提供了三类 RESTAPI 作为控制器的北向接口。**第一类**是面向运营商的北向接口，可以通过 Ryu 控制器中的组件配置管理 SDN 网络。**第二类**是面向 OpenStack 平台的 RESTAPI，主要为 OpenStack 平台的 Quantum 组件提供服务，使得网络资源、计算资源和存储资源被统一交付给用户，实现全资源的池化和按需调度。**第三类**是面向用户的自定义 RESTAPI，Ryu 使用 WSGIWeb 服务器来创建 RESTAPI，使得用户可以使用 REST 或 RPC 定义自己的应用。

3.RESTCONF 协议

RESTCONF 协议以 HTTP 作为传输协议，因此采用 HTTP 报文实体表示消息。

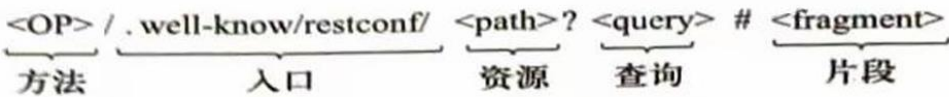


图 6.15 RESTCONF 消息格式

表 6.4 HTTP 请求消息头及描述

名 称	描 述
Accept	可接收的相应内容类型（例如，text/html）
Content-Type	请求消息体的类型（例如，application/json）
Host	服务主机地址
If-Match	如果实体与 ETag 匹配才生效
If-Modified-Since	在规定时间内后的行为被修改才生效
If-Unmodified-Since	在规定时间内后的行为不被修改才生效

表 6.5 HTTP 应答消息头及描述

名 称	描 述
Allow	定义了返回 405 错误时的合法行为（例如，GET）
Content-Type	响应消息体的类型（例如，application/json）
Date	发送消息的时间，包含日期和时分秒
ETag	一个标示符号用于指定资源的特定版本
Last-Modified	某一资源的最后修改时间，包含日期和时分秒
Location	用于指定一个新创建的资源

这是一个基于 REST 模式、用于网络配置与管理的协议，目的在于为 Web 应用提供获取配置数据、状态数据、通知事件的标准机制。因此，该协议以 HTTP 作为传输协议，使用 YANG 语言来定义数据结构，并使用 NETCONF 定义数据存储。

4.题目:

REST API 中, PUT 动词用于完整更新资源, PATCH 用于部分更新资源。



RESTCONF 协议的出现是为了取代 NETCONF 协议。



RESTCONF协议的出现,并非是为了取代NETCONF协议,而是提供一种额外的简易接口,遵循RESTful原则并且与面向资源的设备抽象兼容。与NETCONF相比,RESTCONF提供的功能较简单,有一些功能它是不提供的,例如不提供对数据库的加锁操作,不支持数据库备份。因此,当需要提供复杂功能时,仍然需要使用NETCONF。

ONF 的 NBI 工作组引入的“维度”概念中,横向维度(宽度)表示的是?

- ☐ A 网络技术类型
- ☒ B 接口适用范围与面向功能
- ☐ C 数据存储层级
- ☐ D 抽象等级

ONF 的 NBI(北向接口)工作组提出的“维度”概念分为横向(宽度)和纵向(深度)两个方向:

- **横向维度(宽度):** 描述接口的**适用范围与面向功能**,比如接口是用于网络拓扑管理、流量调度,还是故障监控等,体现了接口覆盖的功能场景广度。
- **纵向维度(深度):** 描述接口的**抽象等级**,从底层设备的具体操作到高层业务的抽象调用,体现了接口对网络资源的抽象程度。

基于意图的北向接口的描述包含对象、操作和结果三个核心要素。

REST API 中,用于创建资源的 HTTP 动词是?

- ☐ A PUT
- ☒ B POST
- ☐ C DELETE
- ☐ D GET

RESTCONF 协议中，数据存储资源可以通过 GET 方法查询。



RESTCONF 协议的传输协议是？

- ☐ (A) NETCONF
- ☐ (B) UDP
- ☒ (C) HTTP
- ☐ (D) TCP

REST API 的 HTTP 响应状态码中，4xx 系列表示的是？

- ☐ (A) 服务器错误
- ☒ (B) 客户端错误
- ☐ (C) 操作成功
- ☐ (D) 重定向

HTTP 状态码按首位数字分为五大类，在 REST API 场景中：

- **2xx 系列：**操作成功（如 200 OK、201 Created）
- **3xx 系列：**重定向（如 301 Moved Permanently、302 Found）
- **4xx 系列：**客户端错误（如 400 Bad Request、401 Unauthorized、404 Not Found），表示请求存在语法错误或无法被服务器理解
- **5xx 系列：**服务器错误（如 500 Internal Server Error、503 Service Unavailable），表示服务器在处理请求时发生了错误

RESTful 架构风格是专门为 SDN 场景设计的。



RESTful 架构风格是由 Roy Fielding 在 2000 年提出的一种通用的 Web 服务设计风格，其核心是基于 HTTP 协议的资源操作

（GET/POST/PUT/DELETE 等），最初用于构建互联网上的分布式系统，并非专门为 SDN（软件定义网络）场景设计。

SDN 只是在发展过程中，因其北向接口需要灵活、标准化的交互方式，而选用了 **RESTful** 作为常用的接口实现方式，但这并不代表 RESTful 是为 SDN 专属设计的。